

Grauer_FinalProject

May 3, 2024

```
[ ]: import os
import sys
import numpy as np
import pandas as pd
import tensorflow as tf
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Model
from keras.models import Sequential
from tensorflow.keras.layers import Input, Dense, Dropout, Flatten
import matplotlib.pyplot as plt
from keras.utils import to_categorical
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
```

```
[ ]: !pip install jinja2
```

```
[3]: import jinja2
```

```
[4]: # Load RNA-seq dataset
data = pd.read_csv("final_project_data.csv")
df = pd.DataFrame(data)
df.set_index('Gene ID', inplace=True)

# Extract gene name
name = df['Gene Name']
name_df = pd.DataFrame(name, columns = ['Gene Name'])
name_df
```

```
[4]:
```

Gene ID	Gene Name
ENSG000000000003	TSPAN6
ENSG000000000005	TNMD
ENSG000000000419	DPM1
ENSG000000000457	SCYL3

```

ENSG00000000460  C1orf112
...
ENSG00000281918      NaN
ENSG00000281919      NaN
ENSG00000281920      NaN
ENSG00000281921      NaN
ENSG00000281922      NaN

```

[65217 rows x 1 columns]

```

[5]: # Format dataset
df.drop('Gene Name', axis=1, inplace=True)
df1 = df.transpose()
df1.head()

```

```

[5]: Gene ID      ENSG00000000003  ENSG00000000005  ENSG00000000419  \
SRR5176628          480              5              347
SRR5176629          516              4              351
SRR5176630          443              3              290
SRR5176631          223              0              122
SRR5176632          285              2              288

```

```

Gene ID      ENSG00000000457  ENSG00000000460  ENSG00000000938  \
SRR5176628          191              55              686
SRR5176629          242              67              143
SRR5176630          201              48              205
SRR5176631           68              21              48
SRR5176632          220              71              224

```

```

Gene ID      ENSG00000000971  ENSG00000001036  ENSG00000001084  \
SRR5176628          1300             617              476
SRR5176629           821             758              946
SRR5176630          1194             822              882
SRR5176631           295             266              221
SRR5176632          1272             576              690

```

```

Gene ID      ENSG00000001167  ...  ENSG00000281913  ENSG00000281914  \
SRR5176628          352  ...              0              0
SRR5176629          376  ...              0              0
SRR5176630          378  ...              0              0
SRR5176631          110  ...              0              0
SRR5176632          437  ...              0              0

```

```

Gene ID      ENSG00000281915  ENSG00000281916  ENSG00000281917  \
SRR5176628           0              0              0
SRR5176629           0              0              0
SRR5176630           0              0              0

```

SRR5176631	0	0	0
SRR5176632	0	0	0
Gene ID	ENSG000000281918	ENSG000000281919	ENSG000000281920 \
SRR5176628	3	0	0
SRR5176629	1	0	0
SRR5176630	3	0	0
SRR5176631	0	0	0
SRR5176632	2	0	0

Gene ID	ENSG000000281921	ENSG000000281922
SRR5176628	1	0
SRR5176629	0	0
SRR5176630	1	0
SRR5176631	1	0
SRR5176632	1	0

[5 rows x 65217 columns]

```
[6]: # Load dataset of experiment design info
exp_design = pd.read_csv("experimental_design.csv")
design_df = pd.DataFrame(exp_design)

# Get sample labels
label_col = design_df.iloc[:,3]
label_names = label_col.values.reshape(-1)
```

```
[7]: # Add labels to RNA-seq data
df_labeled = df1
df_labeled.insert(0, 'Label', label_names)
df_labeled.head(5)
```

[7]:	Gene ID	Label	ENSG000000000003 \
	SRR5176628	Crohn's disease	480
	SRR5176629	non inflammatory bowel disease control	516
	SRR5176630	non inflammatory bowel disease control	443
	SRR5176631	Crohn's disease	223
	SRR5176632	non inflammatory bowel disease control	285

Gene ID	ENSG000000000005	ENSG000000000419	ENSG000000000457 \
SRR5176628	5	347	191
SRR5176629	4	351	242
SRR5176630	3	290	201
SRR5176631	0	122	68
SRR5176632	2	288	220

Gene ID	ENSG000000000460	ENSG000000000938	ENSG000000000971 \
---------	------------------	------------------	--------------------

SRR5176628	55	686	1300
SRR5176629	67	143	821
SRR5176630	48	205	1194
SRR5176631	21	48	295
SRR5176632	71	224	1272

Gene ID	ENSG00000001036	ENSG00000001084	...	ENSG00000281913	\
SRR5176628	617	476	...	0	
SRR5176629	758	946	...	0	
SRR5176630	822	882	...	0	
SRR5176631	266	221	...	0	
SRR5176632	576	690	...	0	

Gene ID	ENSG00000281914	ENSG00000281915	ENSG00000281916	\
SRR5176628	0	0	0	
SRR5176629	0	0	0	
SRR5176630	0	0	0	
SRR5176631	0	0	0	
SRR5176632	0	0	0	

Gene ID	ENSG00000281917	ENSG00000281918	ENSG00000281919	\
SRR5176628	0	3	0	
SRR5176629	0	1	0	
SRR5176630	0	3	0	
SRR5176631	0	0	0	
SRR5176632	0	2	0	

Gene ID	ENSG00000281920	ENSG00000281921	ENSG00000281922
SRR5176628	0	1	0
SRR5176629	0	0	0
SRR5176630	0	1	0
SRR5176631	0	1	0
SRR5176632	0	1	0

[5 rows x 65218 columns]

```
[8]: #df2.to_csv('crohns_data_labeled.csv')
```

```
[9]: # Extract labels
labels = df_labeled.iloc[:,0]

# One hot encode labels
y = pd.get_dummies(labels, dtype='int', drop_first=True)

# Convert y to int
y['non inflammatory bowel disease control'] = y['non inflammatory bowel disease_
↳control'].astype(int)
```

```
[10]: # Get features
features = df1.iloc[:, 1:]

# Normalize through CPM
library_sizes = features.sum(axis=1)
cpm = features.div(library_sizes, axis=0) * 1e6
cpm.head(5)
```

```
[10]: Gene ID      ENSG000000000003  ENSG000000000005  ENSG000000000419  \
SRR5176628      47.265964           0.492354           34.169353
SRR5176629      42.800787           0.331789           29.114489
SRR5176630      36.686022           0.248438           24.015680
SRR5176631      56.339423           0.000000           30.822465
SRR5176632      27.592192           0.193629           27.882636

Gene ID      ENSG000000000457  ENSG000000000460  ENSG000000000938  \
SRR5176628      18.807915           5.415892           67.550941
SRR5176629      20.073237           5.557467           11.861458
SRR5176630      16.645351           3.975009           16.976601
SRR5176631      17.179734           5.305506           12.126871
SRR5176632      21.299236           6.873844           21.686495

Gene ID      ENSG000000000971  ENSG000000001036  ENSG000000001084  \
SRR5176628      128.011987        60.756458           46.872081
SRR5176629      68.099702        62.874025           78.468110
SRR5176630      98.878352        68.072031           73.040793
SRR5176631      74.529730        67.203079           55.834137
SRR5176632      123.148310        55.765272           66.802149

Gene ID      ENSG000000001167  ...  ENSG000000281913  ENSG000000281914  \
SRR5176628      34.661707  ...           0.0           0.0
SRR5176629      31.188171  ...           0.0           0.0
SRR5176630      31.303197  ...           0.0           0.0
SRR5176631      27.790747  ...           0.0           0.0
SRR5176632      42.308028  ...           0.0           0.0

Gene ID      ENSG000000281915  ENSG000000281916  ENSG000000281917  \
SRR5176628           0.0           0.0           0.0
SRR5176629           0.0           0.0           0.0
SRR5176630           0.0           0.0           0.0
SRR5176631           0.0           0.0           0.0
SRR5176632           0.0           0.0           0.0

Gene ID      ENSG000000281918  ENSG000000281919  ENSG000000281920  \
SRR5176628      0.295412           0.0           0.0
SRR5176629      0.082947           0.0           0.0
SRR5176630      0.248438           0.0           0.0
```

SRR5176631	0.000000	0.0	0.0
SRR5176632	0.193629	0.0	0.0

Gene ID	ENSG00000281921	ENSG00000281922
SRR5176628	0.098471	0.0
SRR5176629	0.000000	0.0
SRR5176630	0.082813	0.0
SRR5176631	0.252643	0.0
SRR5176632	0.096815	0.0

[5 rows x 65217 columns]

```
[11]: # Scale to range 0-1
scaler = MinMaxScaler()
features_scaled = pd.DataFrame(scaler.fit_transform(cpm), columns=cpm.columns)
features_scaled.head()
```

```
[11]: Gene ID  ENSG000000000003  ENSG000000000005  ENSG000000000419  ENSG000000000457  \
0          0.269441          0.166297          0.660462          0.542836
1          0.239658          0.112065          0.486175          0.608047
2          0.198873          0.083912          0.310373          0.431385
3          0.329960          0.000000          0.545065          0.458925
4          0.138219          0.065400          0.443702          0.671231
```

Gene ID	ENSG000000000460	ENSG000000000938	ENSG000000000971	ENSG000000001036	\
0	0.390438	0.179619	0.469584	0.457145	
1	0.401382	0.022915	0.233886	0.488480	
2	0.279055	0.037308	0.354971	0.565398	
3	0.381905	0.023662	0.259182	0.552539	
4	0.503140	0.050562	0.450450	0.383287	

Gene ID	ENSG00000001084	ENSG00000001167	...	ENSG00000281913	\
0	0.323178	0.584488	...	0.0	
1	0.616349	0.504769	...	0.0	
2	0.565990	0.507409	...	0.0	
3	0.406335	0.426796	...	0.0	
4	0.508104	0.759975	...	0.0	

Gene ID	ENSG00000281914	ENSG00000281915	ENSG00000281916	ENSG00000281917	\
0	0.0	0.0	0.0	0.0	
1	0.0	0.0	0.0	0.0	
2	0.0	0.0	0.0	0.0	
3	0.0	0.0	0.0	0.0	
4	0.0	0.0	0.0	0.0	

Gene ID	ENSG00000281918	ENSG00000281919	ENSG00000281920	ENSG00000281921	\
0	0.031122	0.0	0.0	0.200611	

1	0.008739	0.0	0.0	0.000000
2	0.026173	0.0	0.0	0.168712
3	0.000000	0.0	0.0	0.514701
4	0.020399	0.0	0.0	0.197237

Gene ID	ENSG00000281922
0	0.0
1	0.0
2	0.0
3	0.0
4	0.0

[5 rows x 65217 columns]

```
[13]: # Function to perform Random Forest classification and get most influential
      ↪ genes
def rf_gene_selection(num_genes):

    y_array = y['non inflammatory bowel disease control'].astype(int).values

    # Train RF
    rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
    rf_model.fit(features_scaled, y_array)

    # Get feature importances
    feature_importances = rf_model.feature_importances_

    # Sort importances from most to least important
    sorted_indices = np.argsort(feature_importances)[::-1]
    top_gene_indices = sorted_indices[:num_genes]

    # Get IDs of top gene indices
    top_gene_ids = features_scaled.columns[top_gene_indices]

    # Get names of top genes
    top_genes_list = top_gene_ids.tolist()
    top_gene_names = name[top_genes_list]

    # Convert to df
    #top_genes_df = pd.DataFrame(top_gene_names)

    # Create feature set with selected top genes
    X_selected = features_scaled[top_genes_list]

    return X_selected

# Function to create a NN
```

```

def create_nn(input_dim):
    model = Sequential()
    model.add(Dense(64, activation='relu', input_shape=(input_dim,)))
    #model.add(Dense(32, activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    model.compile(optimizer='adam', loss='binary_crossentropy',
    ↪metrics=['accuracy'])
    return model

```

```

[14]: # Range of number of top genes to try
numbers = [300,275,250,225,200,175,150,125,100,75,50,25,1]

# Keep track of accuracy for each
acc_scores = []
acc_dict = {}

# Perform k-fold cross-validation to find optimal number of top genes
for n in numbers:

    # Use RF to get feature set with selected number of top genes
    X = rf_gene_selection(n)

    # Conduct 10 trials
    accuracies = []

    for i in range(3):
        # Split into train and test
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.
        ↪2, random_state=42)

        # Create and train NN
        nn_model = create_nn(input_dim=n)
        hist = nn_model.fit(X_train, y_train, epochs=50, batch_size=32,
        ↪validation_data=(X_test, y_test), verbose=0)

        # Evaluate the model on test data
        test_loss, test_accuracy = nn_model.evaluate(X_test, y_test)
        accuracies.append(test_accuracy)

    # Find average accuracy across all trials
    avg_accuracy = np.mean(accuracies)
    print(str(n) + ": " + str(avg_accuracy))

    # Store accuracy
    acc_scores.append(avg_accuracy)
    acc_dict[n] = avg_accuracy

```

```

# Find the index of the maximum average accuracy score
optimal_n_index = np.argmax(acc_scores)
optimal_n = numbers[optimal_n_index]

# Print the optimal number of top genes
print(f"Optimal number of top genes: {optimal_n}")

```

```

2/2 [=====] - 0s 4ms/step - loss: 0.1127 - accuracy:
0.9388
2/2 [=====] - 0s 4ms/step - loss: 0.1206 - accuracy:
0.9796
2/2 [=====] - 0s 5ms/step - loss: 0.1307 - accuracy:
0.9388
300: 0.9523809750874838
2/2 [=====] - 0s 5ms/step - loss: 0.1271 - accuracy:
0.9592
2/2 [=====] - 0s 6ms/step - loss: 0.1095 - accuracy:
0.9592
2/2 [=====] - 0s 4ms/step - loss: 0.1110 - accuracy:
0.9796
275: 0.965986410776774
2/2 [=====] - 0s 4ms/step - loss: 0.1280 - accuracy:
0.9388
2/2 [=====] - 0s 5ms/step - loss: 0.1322 - accuracy:
0.9184
2/2 [=====] - 0s 4ms/step - loss: 0.1421 - accuracy:
0.9388
250: 0.9319728016853333
2/2 [=====] - 0s 5ms/step - loss: 0.1357 - accuracy:
0.9388
2/2 [=====] - 0s 5ms/step - loss: 0.1777 - accuracy:
0.9184
2/2 [=====] - 0s 5ms/step - loss: 0.1302 - accuracy:
0.9592
225: 0.9387755195299784
2/2 [=====] - 0s 5ms/step - loss: 0.1410 - accuracy:
0.9388
2/2 [=====] - 0s 5ms/step - loss: 0.1266 - accuracy:
0.9388
2/2 [=====] - 0s 4ms/step - loss: 0.1170 - accuracy:
0.9592
200: 0.9455782572428385
2/2 [=====] - 0s 5ms/step - loss: 0.1416 - accuracy:
0.9184
2/2 [=====] - 0s 4ms/step - loss: 0.1439 - accuracy:
0.9184
2/2 [=====] - 0s 5ms/step - loss: 0.1435 - accuracy:

```

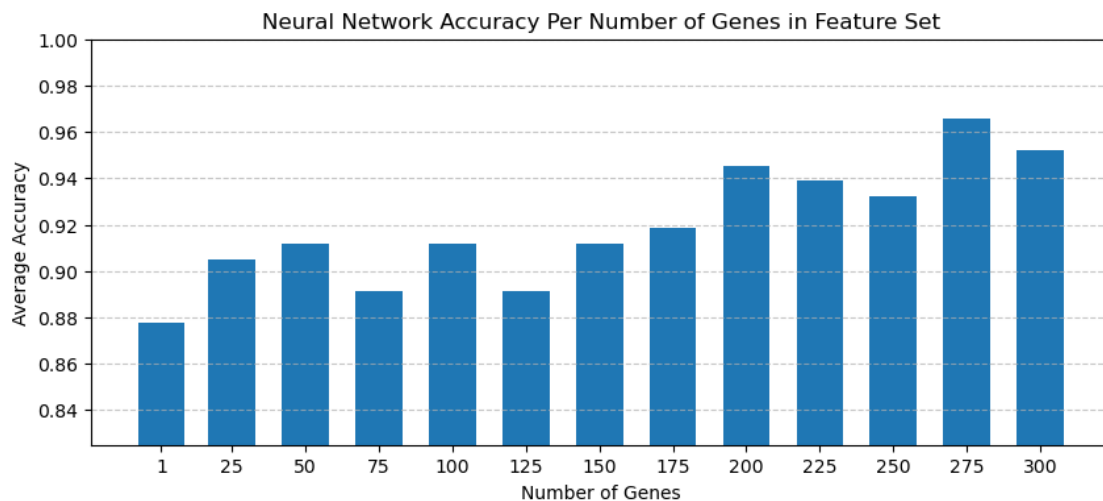
0.9184
 175: 0.918367326259613
 2/2 [=====] - 0s 5ms/step - loss: 0.1612 - accuracy:
 0.9184
 2/2 [=====] - 0s 6ms/step - loss: 0.1673 - accuracy:
 0.8980
 2/2 [=====] - 0s 5ms/step - loss: 0.1607 - accuracy:
 0.9184
 150: 0.9115646084149679
 2/2 [=====] - 0s 4ms/step - loss: 0.1950 - accuracy:
 0.8776
 2/2 [=====] - 0s 4ms/step - loss: 0.1734 - accuracy:
 0.8776
 2/2 [=====] - 0s 5ms/step - loss: 0.1746 - accuracy:
 0.9184
 125: 0.8911564548810323
 2/2 [=====] - 0s 45ms/step - loss: 0.1510 - accuracy:
 0.9184
 2/2 [=====] - 0s 4ms/step - loss: 0.1633 - accuracy:
 0.8980
 2/2 [=====] - 0s 10ms/step - loss: 0.1569 - accuracy:
 0.9184
 100: 0.9115646084149679
 2/2 [=====] - 0s 6ms/step - loss: 0.1728 - accuracy:
 0.8980
 2/2 [=====] - 0s 4ms/step - loss: 0.1794 - accuracy:
 0.8980
 2/2 [=====] - 0s 4ms/step - loss: 0.1709 - accuracy:
 0.8776
 75: 0.8911564548810323
 2/2 [=====] - 0s 7ms/step - loss: 0.1770 - accuracy:
 0.9184
 2/2 [=====] - 0s 6ms/step - loss: 0.1732 - accuracy:
 0.9184
 2/2 [=====] - 0s 7ms/step - loss: 0.1835 - accuracy:
 0.8980
 50: 0.9115646084149679
 2/2 [=====] - 0s 9ms/step - loss: 0.1982 - accuracy:
 0.8980
 2/2 [=====] - 0s 9ms/step - loss: 0.1999 - accuracy:
 0.8980
 2/2 [=====] - 0s 5ms/step - loss: 0.2060 - accuracy:
 0.9184
 25: 0.9047618905703226
 2/2 [=====] - 0s 8ms/step - loss: 0.3578 - accuracy:
 0.8776
 2/2 [=====] - 0s 11ms/step - loss: 0.3598 - accuracy:
 0.8776

```
2/2 [=====] - 0s 7ms/step - loss: 0.3595 - accuracy:
0.8776
1: 0.8775510191917419
Optimal number of top genes: 275
```

```
[19]: # Plot average accuracy per number of genes selected through RF
num_values = list(acc_dict.keys())
accuracy = [acc_dict[n] for n in num_values]

plt.figure(figsize=(10, 4))
plt.bar(num_values, accuracy, label='Accuracy', width=16)
plt.xlabel('Number of Genes')
plt.ylabel('Average Accuracy')
plt.title('Neural Network Accuracy Per Number of Genes in Feature Set')
plt.xticks(num_values)
plt.ylim(0.825, 1.0)
plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.show()
```



```
[38]: # Random forest with optimal n number of top genes to create final feature set
X_final = rf_gene_selection(275)
ids_final = X_final.columns.tolist()
names_final = [name_df.loc[id, 'Gene Name'] if id in name_df.index else
               'Unknown' for id in ids_final]

# IDs and names of genes in feature set
final_df = pd.DataFrame({'Gene ID': ids_final, 'Gene Name': names_final})
final_df
```

```
# Data saved to undergo Ingenuity Pathway Analysis (IPA)
#final_df.to_csv('selected_genes.csv')
```

```
[38]:
```

	Gene ID	Gene Name
0	ENSG00000224397	PELATON
1	ENSG00000134470	IL15RA
2	ENSG00000166527	CLEC4D
3	ENSG00000168062	BATF2
4	ENSG00000149929	HIRIP3
..	...	...
270	ENSG00000126467	TSKS
271	ENSG00000268205	NaN
272	ENSG00000109181	UGT2B10
273	ENSG00000169174	PCSK9
274	ENSG00000259450	NaN

```
[275 rows x 2 columns]
```

```
[39]: # Split into train and test
X_train, X_test, y_train, y_test = train_test_split(X_final, y, test_size=0.2,
    random_state=42)

# Build final model
final_model = create_nn(input_dim=275)
final_hist = final_model.fit(X_train, y_train, epochs=50, batch_size=32,
    validation_data=(X_test, y_test), verbose=0)

# Evaluate the model on test data
test_loss, test_accuracy = final_model.evaluate(X_test, y_test)
print("Test Loss: " + str(test_loss))
print("Test Accuracy:" + str(test_accuracy))
```

```
2/2 [=====] - 0s 5ms/step - loss: 0.0995 - accuracy: 0.9796
```

```
Test Loss: 0.09946562349796295
```

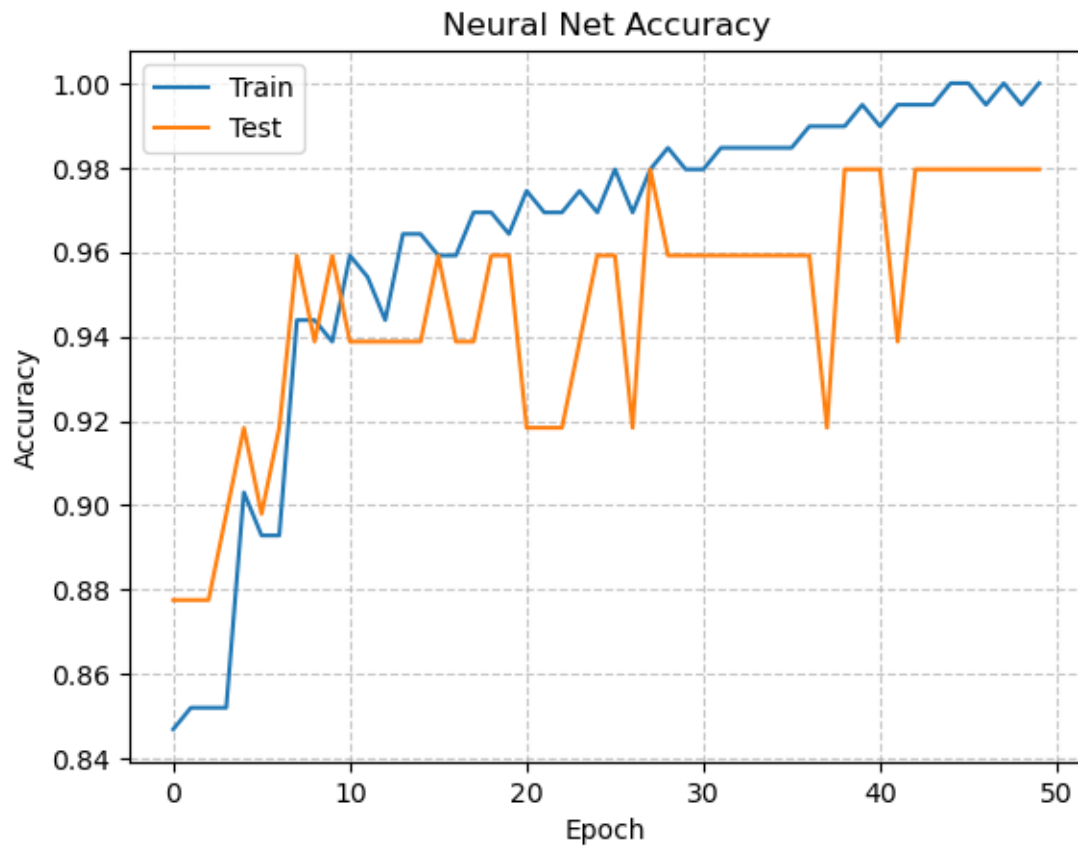
```
Test Accuracy:0.9795918464660645
```

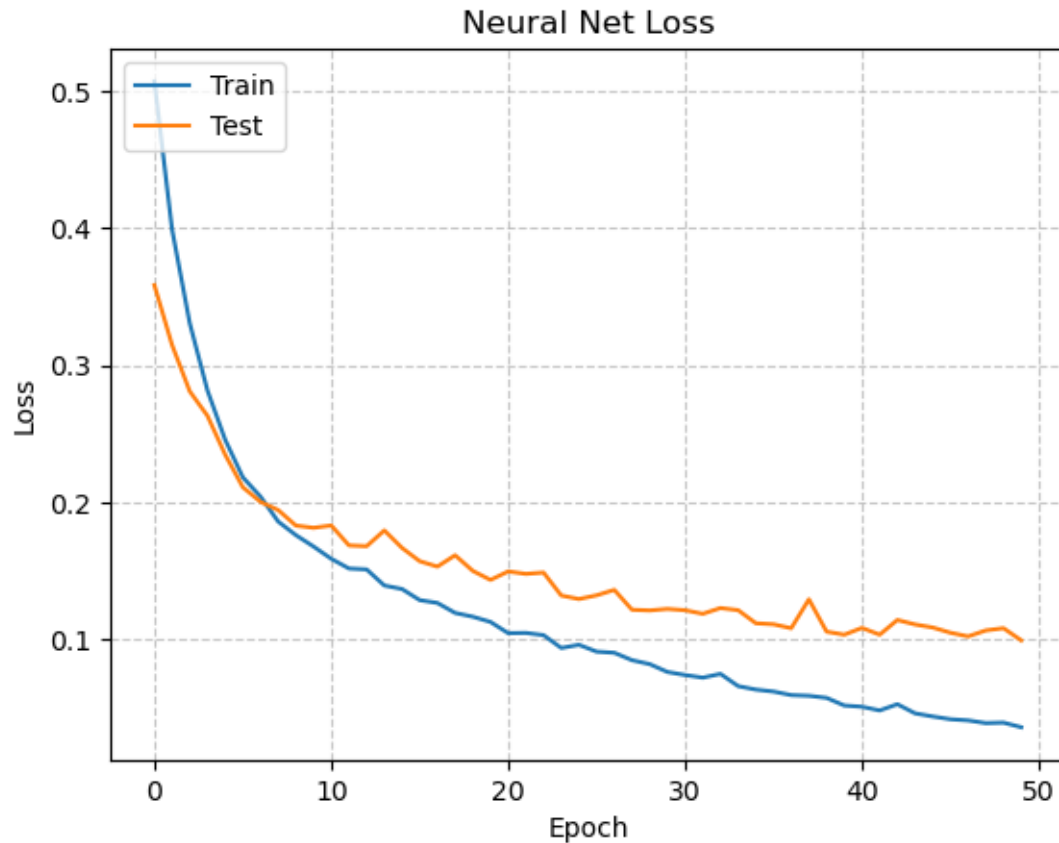
```
[41]: # Plot training & validation accuracy values
plt.plot(final_hist.history['accuracy'])
plt.plot(final_hist.history['val_accuracy'])
plt.title('Neural Net Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.legend(['Train', 'Test'], loc='upper left')
plt.show()
```

```

# Plot training & validation loss values
plt.plot(final_hist.history['loss'])
plt.plot(final_hist.history['val_loss'])
plt.title('Neural Net Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.legend(['Train', 'Test'], loc='upper left')
plt.show()

```





```
[134]: # Predict on test data
pred_prob = final_model.predict(X_test)
pred = pred_prob.round().astype(int)
pred_df = pd.DataFrame(pred, columns=['Pred Label'])

# Make results dataframe
y_test_df = pd.DataFrame(y_test.values, columns=['True Label'])
pred_df.reset_index(drop=True, inplace=True)
y_test_df.reset_index(drop=True, inplace=True)
results = pd.concat([pred_df, y_test_df], axis=1)
results.head()
```

2/2 [=====] - 0s 7ms/step

```
[134]:
```

	Pred Label	True Label
0	0	0
1	0	0
2	1	1
3	0	0
4	0	0

```
[135]: # Classification report`
print(classification_report(y_test, pred))
```

	precision	recall	f1-score	support
0	1.00	0.98	0.99	43
1	0.86	1.00	0.92	6
accuracy			0.98	49
macro avg	0.93	0.99	0.96	49
weighted avg	0.98	0.98	0.98	49

```
[164]: # Feature Importance
input_tensor = tf.convert_to_tensor(X_test, dtype=tf.float32)

with tf.GradientTape() as tape:
    tape.watch(input_tensor)
    output = final_model(input_tensor)

gradients = tape.gradient(output, input_tensor)

# Calculate feature importance
feature_importance = np.mean(np.abs(gradients.numpy()), axis=0)
feature_importance = np.nan_to_num(feature_importance, nan=0.0)

# Get top 10 most influential genes
top_n = 10
top_indices = np.argsort(feature_importance)[::-1][:top_n]
top_nn_genes = X_final.columns[top_indices]
importances = []
top_nn_ids = []

for i, gene_id in enumerate(top_genes):
    importance_score = feature_importance[top_indices[i]]
    importances.append(importance_score)
    top_nn_ids.append(gene_id)

top_gene_names = [name_df.loc[gene_id, 'Gene Name'] if gene_id in name_df.index
↳ else 'Unknown' for gene_id in top_nn_genes]
top_nn_genes_df = pd.DataFrame({'Gene ID': top_nn_ids, 'Gene Name':
↳ top_gene_names, 'Importance Score': importances})
top_nn_genes_df.style.set_caption("Top 10 Influential Genes in NN Prediction")
```

```
[164]: <pandas.io.formats.style.Styler at 0x17ce6e290>
```